

Phase 5: Apex Programming (Developer) –

Project: Consumer Goods Distribution Partner Portal

Overview

Phase 5 focuses on **Apex Programming** to extend Salesforce functionality through custom classes, triggers, batch processes, and scheduled jobs. Apex enables developers to implement business logic that goes beyond standard Salesforce automation.

This phase includes:

- Creation of Apex Classes, Triggers, and Custom Exception Classes
- Implementation of Batch, Queueable, and Scheduled Apex
- Use of SOQL for data retrieval and DML operations
- Adherence to trigger design patterns for maintainability and scalability

1. Apex Components Created

1.1 Apex Classes

Name	Description
OrderUtility	Utility class for common order-related operations
OrderProcessor	Processes orders with business logic
PaymentProcessor	Handles payment transactions

1.2 Apex Triggers

Name	Object	Purpose
OrderTrigger	Order__c	Executes before insert/update to validate and process orders
PaymentTrigger	Payment__c	Executes after insert/update to manage payment status

1.3 Custom Exception Classes

Name	Purpose
OrderProcessingException	Handles errors during order processing
PaymentException	Handles payment-related errors

1.4 Batch Apex Classes

Name	Purpose
OrderBatchProcessor	Processes large sets of orders asynchronously

1.5 Queueable Apex Classes

Name	Purpose
OrderQueueable	Handles asynchronous processing of order operations

1.6 Scheduled Apex Classes

Name	Purpose
ScheduledOrderProcessor	Executes batch or queueable Apex on a scheduled basis

2. Step-by-Step Instructions to Create Apex Components

2.1 Creating an Apex Class

1. Navigate to **Setup** → **Apex Classes** → **New**.
2. Enter the class name (e.g., OrderUtility).
3. Paste the provided code snippet.
4. Click **Save**.

Developer Console - Google Chrome

orgfarm-42cc4ddc20-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage

File Edit Debug Test Workspace Help < >

OrderUtility.apxc OrderTrigger.apxt OrderHandler.apxc Order_Trigger.apxt OrderBatchProcessor.apxc OrderQueueal

Code Coverage: None API Version: 64

```
1 public class OrderUtility {
2     public String orderNumber;
3     public Decimal orderAmount;
4
5     // Constructor
6     public OrderUtility(String orderNumber, Decimal orderAmount) {
7         this.orderNumber = orderNumber;
8         this.orderAmount = orderAmount;
9     }
10
11     // Method to apply discount
12     public Decimal applyDiscount(Decimal discountPercent) {
13         return orderAmount - (orderAmount * discountPercent / 100);
14     }
15 }
```

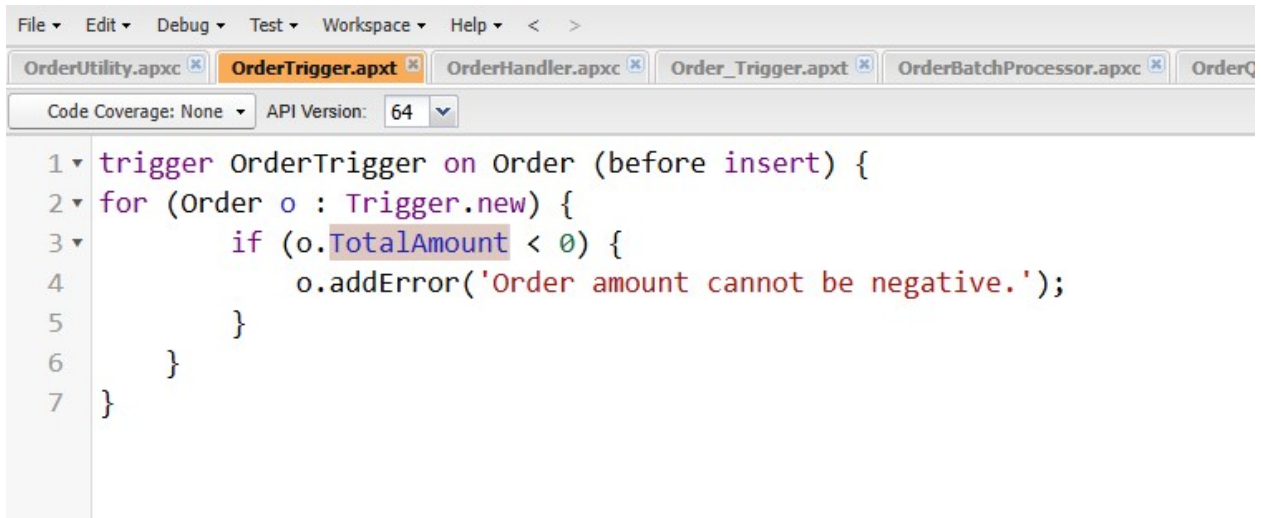
Apex Class: OrderUtility

```
public class OrderUtility {
    // Utility method to calculate total amount
    public static Decimal calculateTotal(List<Order__c> orders) {
        Decimal total = 0;
        for(Order__c ord : orders) {
            total += ord.Quantity__c * ord.UnitPrice__c;
        }
        return total;
    }

    // Utility method to validate order
    public static void validateOrder(Order__c order) {
        if(order.Quantity__c <= 0) {
            throw new OrderProcessingException('Quantity must be greater than
zero.');
```

2.2 Creating an Apex Trigger

1. Navigate to **Setup** → **Apex Triggers** → **New**.
2. Select the object (e.g., Order__c).
3. Enter the trigger name (e.g., OrderTrigger).
4. Paste the trigger code snippet.
5. Click **Save**.



```
File Edit Debug Test Workspace Help < >
OrderUtility.apxc OrderTrigger.apxt OrderHandler.apxc Order_Trigger.apxt OrderBatchProcessor.apxc OrderQ
Code Coverage: None API Version: 64
1 trigger OrderTrigger on Order (before insert) {
2   for (Order o : Trigger.new) {
3     if (o.TotalAmount < 0) {
4       o.addError('Order amount cannot be negative.');
5     }
6   }
7 }
```

Apex Trigger: OrderTrigger

```
trigger OrderTrigger on Order__c (before insert, before update) {
    if(Trigger.isBefore) {
        if(Trigger.isInsert || Trigger.isUpdate) {
            OrderTriggerHandler.handleBeforeInsertUpdate(Trigger.new);
        }
    }
}
```

orgfarm-42cc4ddc20-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage

File Edit Debug Test Workspace Help < >

OrderUtility.apxc OrderTrigger.apxt **OrderHandler.apxc** Order_Trigger.apxt OrderBatchProcessor.apxc OrderQueueable.apxc ScheduledOrderProcessor.apxc

Code Coverage: None API Version: 64

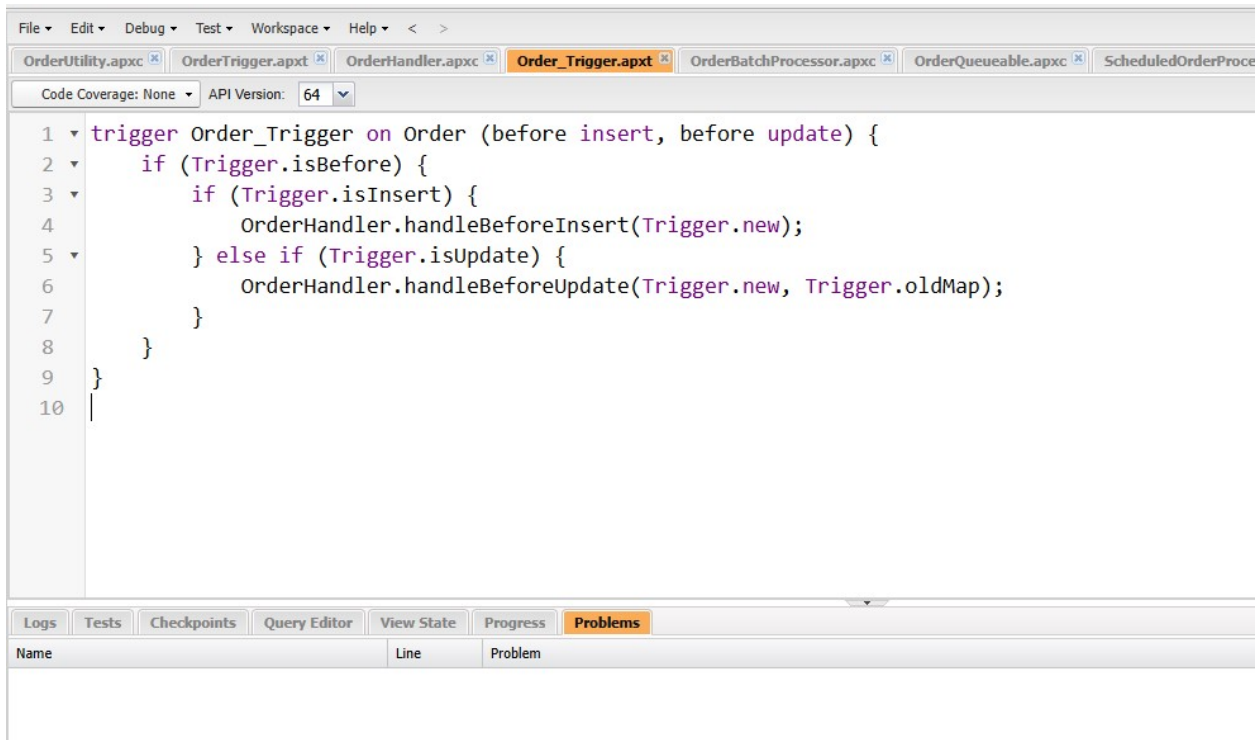
```
1 public class OrderHandler {
2     public static void handleBeforeInsert(List<Order> newOrders)
3     {
4         for (Order order : newOrders)
5         {
6             if (order.TotalAmount < 0) {
7                 order.addError('Order amount cannot be negative.');
8             }
9         }
10    }
11    public static void handleBeforeUpdate(List<Order> newOrders, Map<Id, Order> oldMap) {
12        // Update logic here
13    }
14 }
15
```

Logs Tests Checkpoints Query Editor View State Progress **Problems**

Name	Line	Problem
------	------	---------

Trigger Handler: OrderTriggerHandler

```
public class OrderTriggerHandler {
    public static void handleBeforeInsertUpdate(List<Order__c> orders) {
        for(Order__c ord : orders) {
            OrderUtility.validateOrder(ord);
            ord.TotalAmount__c = OrderUtility.calculateTotal(new
List<Order__c>{ord});
        }
    }
}
```



Explanation: Using a handler class ensures **separation of concerns**, making triggers easier to maintain.

2.3 Creating Custom Exception Classes

Apex Class: OrderProcessingException

```
public class OrderProcessingException extends Exception {}
```

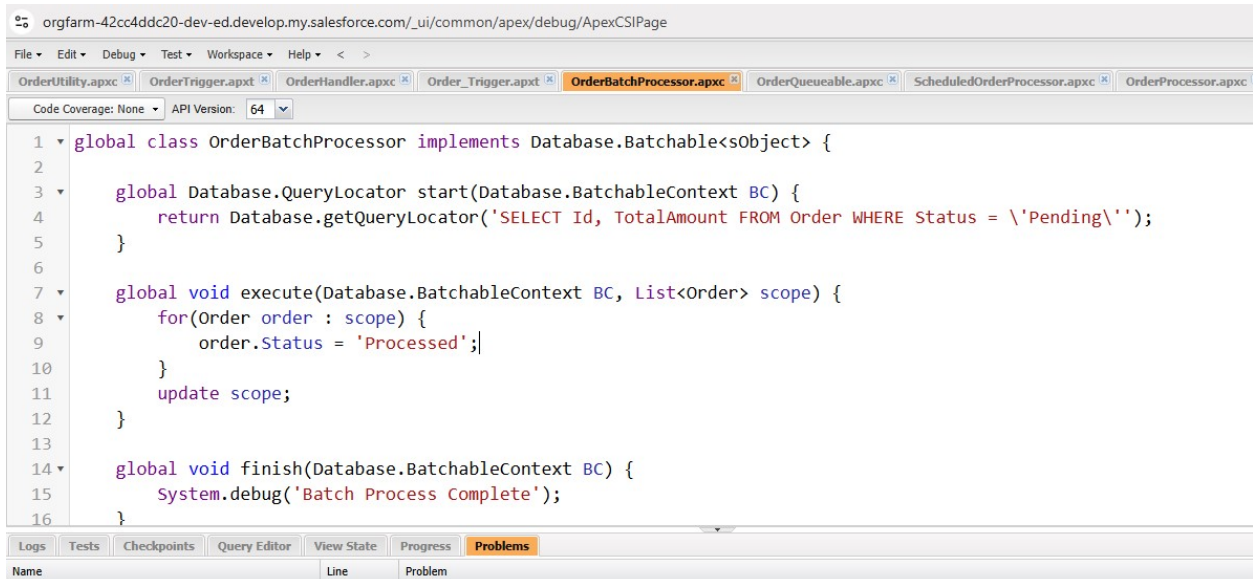
Apex Class: PaymentException

```
public class PaymentException extends Exception {}
```

Explanation: Custom exceptions allow for **clear, controlled error handling** in complex Apex logic.

2.4 Creating a Batch Apex Class

1. Navigate to **Setup** → **Apex Classes** → **New**.
2. Enter the batch class name (e.g., OrderBatchProcessor).
3. Paste the code snippet.
4. Click **Save**.



Apex Class: OrderBatchProcessor

```
global class OrderBatchProcessor implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext BC) {
        return Database.getQueryLocator('SELECT Id, Quantity__c, UnitPrice__c
FROM Order__c WHERE Status__c = \'Pending\''');
    }

    global void execute(Database.BatchableContext BC, List<Order__c> scope) {
        for(Order__c ord : scope) {
            ord.TotalAmount__c = ord.Quantity__c * ord.UnitPrice__c;
        }
        update scope;
    }

    global void finish(Database.BatchableContext BC) {
        System.debug('Batch Processing Completed.');
```

2.5 Creating a Queueable Apex Class

Apex Class: OrderQueueable

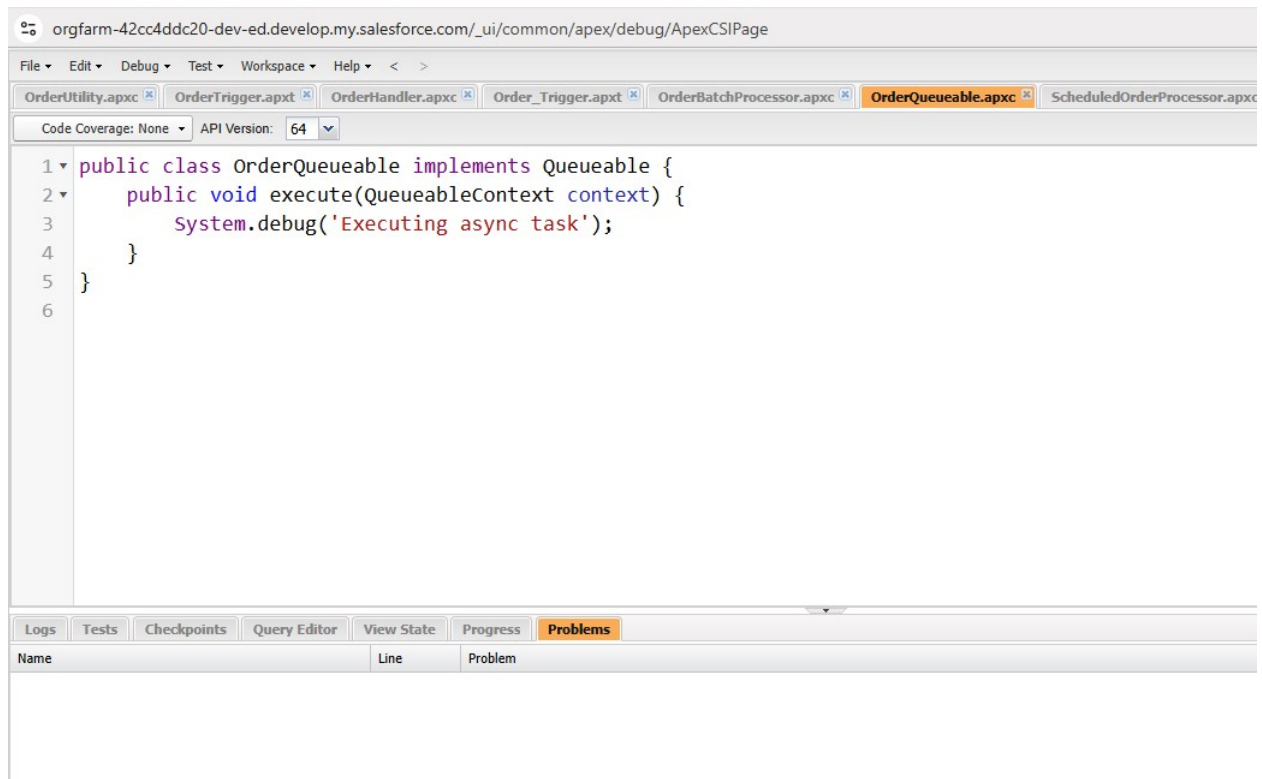
```
public class OrderQueueable implements Queueable {
    private List<Order__c> orders;

    public OrderQueueable(List<Order__c> orders) {
        this.orders = orders;
    }
}
```

```

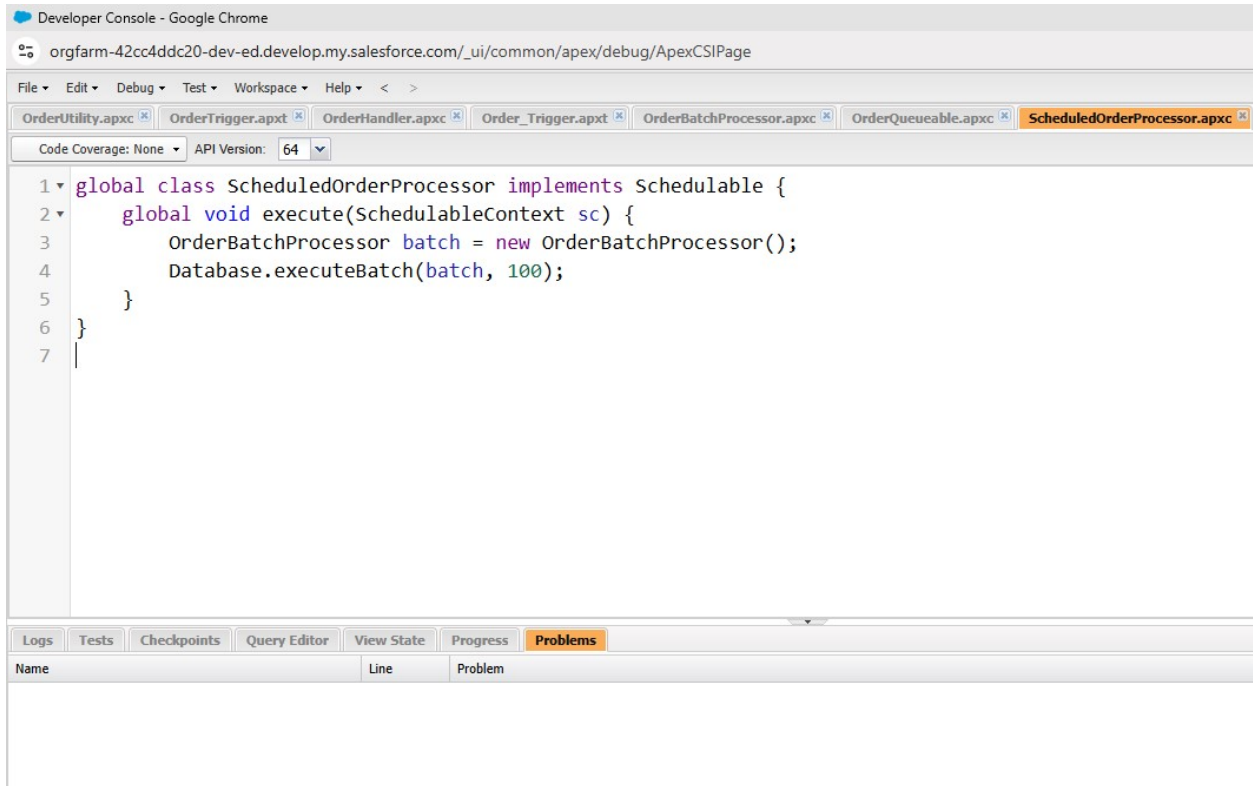
    public void execute(QueueableContext context) {
        for(Order__c ord : orders) {
            ord.TotalAmount__c = ord.Quantity__c * ord.UnitPrice__c;
        }
        update orders;
    }
}

```



2.6 Creating a Scheduled Apex Class

1. Navigate to **Setup** → **Apex Classes** → **New**.
2. Enter the class name (e.g., `ScheduledOrderProcessor`).
3. Implement `Schedulable` interface.
4. Save and schedule via **Setup** → **Apex Classes** → **Schedule Apex**.



Apex Class: ScheduledOrderProcessor

```
public class ScheduledOrderProcessor implements Schedulable {
    public void execute(SchedulableContext SC) {
        OrderBatchProcessor batch = new OrderBatchProcessor();
        Database.executeBatch(batch, 50);
    }
}
```

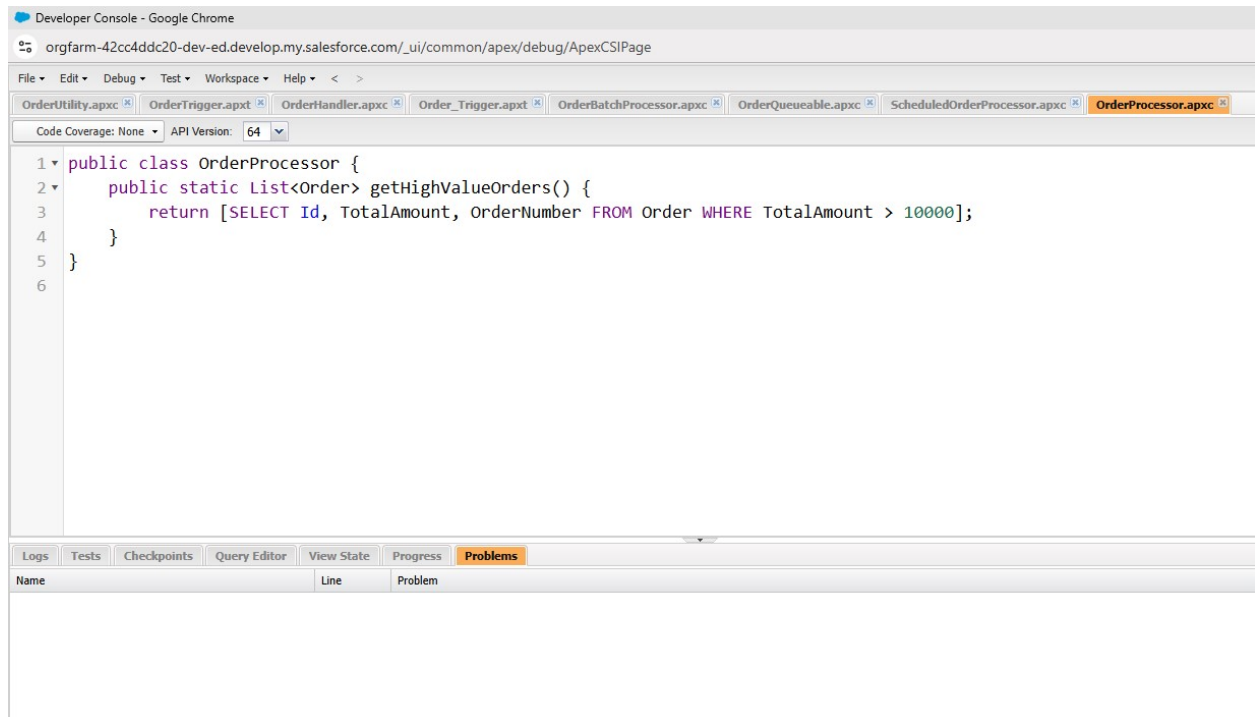
2.7 Using SOQL in Apex

- Use **SOQL** to retrieve Salesforce data within Apex classes.

- Example in a batch or queueable class:

```
List<Order__c> pendingOrders = [SELECT Id, Quantity__c, UnitPrice__c FROM
Order__c WHERE Status__c = 'Pending'];
```

- Use **DML operations** to insert, update, or delete records after processing.



3. Summary

The Apex components implemented in Phase 5 provide:

- **Automation:** Automates order processing, payments, and batch operations.
- **Error Handling:** Custom exceptions provide controlled error management.
- **Scalability:** Batch and Queueable Apex handle large datasets asynchronously.
- **Maintainability:** Trigger handler patterns separate logic from triggers for easier updates.
- **Scheduling:** Scheduled Apex ensures timely processing of recurring tasks.

Together, these components **enhance the Salesforce system**, making it more robust, efficient, and capable of handling complex business logic seamlessly.